BUILD A LARGE LANGUAGE MODEL

Capítulo 5 - Pretraining on unlabeled data (Pré treinamento com dados não rotulados)

Anotações:

- Implementação do loop de treinamento com modelo de avaliação de métricas e carregamento de pesos pré treinados no modelo.
- **Weight parameters (Parâmetros de peso):** Pesos são os parâmetros treináveis que o processo de aprendizagem ajusta. No PyTorch os pesos são armazenados em camadas lineares, eles foram utilizados na implementação do multi-head attention e do modelo GPT. A biblioteca também permite direto acesso para todos os parâmetros treináveis do modelo.
- Utiliza-se um tokenizer para codificar o texto de entrada em token ID e dado X token ID o modelo irá produzir 4 vetores logit (linhas) onde cada vetor tem 50257 elementos (colunas). Após isso, serão convertidos os logits em tokens ID e utilizado o tokenizer para decodificar os IDs de volta para representações textuais. Isso completa o ciclo de entrada de texto para saída de texto.
- **Logits:** vetores representando a probabilidade de distribuição para cada token no vocabulário.
- O que define o que faz o texto coerente e com qualidade é a implementação de um método numérico para avaliar o conteúdo gerado.
- **Métrica de loss/perda:** Calcula-se a perda de geração de texto. Acessa numericamente a qualidade do texto gerado durante o treinamento.
- **Avaliação do modelo de geração de texto:** Usa vocabulário para mapear a entrada de texto em token IDs, tem vocabulário_tamanho linhas de vetores probabilísticos para cada token (softmax), localiza a posição do index com maior valor de probabilidade em cada linha do vetor (argmax), obtém todos os tokens IDs previstos e mapeia as posições de index de volta para texto.
- **Targets:** são as entradas mas deslocadas uma posição à frente. Isso é essencial para ensinar o modelo a prever o próximo token em uma sequência.
- **Processo de avaliação de texto:** Medir o quão longe os tokens gerados estão das predições corretas (alvos).
- **Modelo de treinamento:** Mira em aumentar a probabilidade da softmax nas posições de index que corresponde ao alvo token IDs correto. O objetivo é maximizar os valores que correspondem ao index do token no vetor de alvo.
- **Probabilidades iniciais:** giram em torno de 1/num vocabulário.
- Um modelo não treinado produz vetores aleatórios para cada token.
- **Backpropagation:** Método usado para fazer os ajuste de pesos, o modelo olha pro resultado final e vai ajustando de trás para frente para não errar de novo na próxima vez e para o

backpropagation saber como ajustar ele precisa saber o tamanho do erro e a função de perda/loss mede exatamente a distância entre a resposta correta e o “chute” que o modelo deu.

- **Cross entropy loss:** Fórmula matemática mais famosa na Inteligência Artificial para calcular aquele erro (loss). Ela mede exatamente a distância entre o "chute" do modelo (as porcentagens que ele deu para cada palavra) e a realidade (a palavra 100% correta). Calcular a Cross Entropy é a mesma coisa que calcular os logaritmos das probabilidades só coloca um sinal negativo na frente para transformar isso em uma nota de erro positiva.
- **Perplexity:** É outra forma de medir o erro do modelo, junto com o loss. Assim como na perda, quanto menor esse número, melhor o modelo. Ela traduz a matemática complexa do loss para o tamanho da confusão do modelo. Fica muito mais fácil para nós entendermos o quão perdida ela está.
- Para preparar o treino do modelo, primeiro dividimos o texto original em duas partes uma para o treinamento e outra para a validação, que funcionará como uma prova para testar o que foi aprendido. Em seguida, pegamos a parte do treinamento e transformamos as palavras em tokens. Como o texto inteiro é muito grande, nós o cortamos em fatias menores de tamanho igual, formando blocos de texto. Depois disso, embaralhamos todas essas fatias para garantir que o modelo realmente aprenda os padrões em vez de apenas decorar a ordem das frases. Por fim, organizamos esses blocos misturados em pequenos pacotes agrupados, chamados de *batches*, para enviá-los para o modelo começar a treinar. Na prática, o modelo pode se beneficiar ao treinar a LLM com comprimentos variáveis de entradas para ajudar e melhor generalizar diferentes tipos de entradas quando for usado o modelo.
- A cada pacote lido, o sistema calcula o tamanho do erro da IA para entender o quanto ela "chutou" errado. Com base nessa nota vermelha, o sistema descobre a direção exata de onde ela se confundiu e usa essa informação para ajustar as "engrenagens" internas do modelo. Esse processo de errar, entender o erro e reajustar a máquina se repete inúmeras vezes com um único objetivo fazer o modelo aprender continuamente para que a sua taxa de erro vá diminuindo até ficar a menor possível.
- **Treinamento LLM (8 passos):** iterar em cada epoch, processar batches, “ressetar” gradientes, calcular loss e novos gradientes, atualizar pesos e concluir com monitoramentos dos passos como printar as losses e gerar amostras de texto.
- **AdamW:** É um algoritmo utilizado para ajustar os pesos de uma rede neural durante o treinamento. Ele é uma versão aprimorada do Adam, que utiliza uma técnica chamada Weight Decay para evitar que os pesos do modelo fiquem muito grandes. Isso ajuda a reduzir o risco de overfitting, que acontece quando o modelo aprende muito bem os dados de treinamento, mas tem dificuldade em lidar com dados novos. Dessa forma, ele ajuda o modelo a aprender melhor e a produzir resultados mais eficientes, sendo bastante utilizado no treinamento de modelos LLM.

- **Decoding strategies (Estratégias de geração de texto):** Observa-se 2 estratégias a temperature scaling e top-k sampling. A primeira troca a regra rígida (maior probabilidade) por um "sorteio" baseado nas porcentagens. A palavra com a nota mais alta ainda tem mais chances de ser escolhida, mas outras palavras boas também entram na disputa, como as palavras são "sorteadas" de forma probabilística, a cada vez que fizer a mesma pergunta, ela poderá construir frases diferentes. O segundo combinado com sampling probabilistic e temperature scaling melhora os resultados de geração de texto, nele podemos restringir o tokens de amostra para o top-k mais prováveis tokens e excluir todos os outros tokens do processo de seleção ao mascarar seus scores de probabilidade.
- A arquitetura dos diferentes tamanhos de modelos GPT é a mesma, tirando que diferentes elementos arquiteturais são repetidos por uma quantidade diferente de vezes e o tamanho do embedding difere também.
- **Weight tying:** O GPT-2 utiliza os mesmos pesos da camada de entrada (que transforma as palavras em números) na camada de saída (que escolhe a próxima palavra). Isso significa que ele compartilha os mesmos pesos em duas partes do modelo, em vez de criar pesos diferentes para cada uma.